BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

EEE 460

Optoelectronics Laboratory

Final Project Report

Section: G1 Group: 03

Design and Implementation of a Smart Colorimeter for Automated Concentration Analysis and Chemical Identification

Course Instructors:

Dr Muhammad Anisuzzaman Talukder , Professor
Mushfiquzzaman Abid , Part time Lecturer

Signature of Instructor: _____

Academic Honesty Statement:

IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. *You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.*

"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Signature: Mushfika Ikfat _____ **Full Name: Mushfika Ikfat** **Student ID: 2006017**	**Signature:** Turjo _____ **Full Name: Zulkar Naaen Turjo** **Student ID: 2006021**
Signature: Kawsain _____ **Full Name: Kawsain Bin Salim** **Student ID: 2006028**	**Signature:** Shakib _____ **Full Name: Md Shakib Hossain** **Student ID: 2006065**

Table of Contents

1 Abstract

This project presents a low-cost, **automated Smart Colorimeter** for precise concentration analysis and **chemical identification**. Built with an Arduino, a 6-LED optical array, and a custom Python GUI, the system streamlines optical analysis through three core functions: autonomous database generation via peak absorbance (lambda_max) detection, unknown concentration calculation using a highly accurate algorithm (linear interpolation), and automated chemical dye identification via optical fingerprinting. By integrating real-time baseline calibration and an intuitive software interface, this project provides a robust, accessible, and user-friendly alternative to expensive commercial laboratory equipment.

2 Introduction

2.1 Background and Motivation

Colorimetry is a fundamental optical analytical technique used for measuring the concentration of colored substances in a solution. The core principle relies on the fact that a colored solution absorbs a specific portion of incident light; the darker the color (higher absorbance), the higher the concentration of the chemical. This phenomenon is mathematically governed by the **Beer-Lambert Law**, which establishes a linear relationship between absorbance (A) and concentration (C) as A = € * C* L, where € is the molar extinction coefficient and L is the path length.

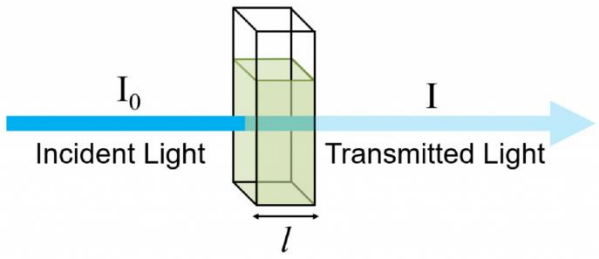


Figure *Basic working principle of a colorimeter demonstrating incident light (I_0) passing through a sample cuvette to yield transmitted light (I).*

While traditional colorimeters are reliable, they often present several limitations. They typically require manual filter swapping for different wavelengths, lack real-time automated data processing, and are prohibitively expensive for widespread use in educational laboratories. To bridge this gap, this project focuses on the **design and implementation of a Smart Colorimeter** that automates the analytical process while maintaining a low-cost architecture.

2.2 Proposed System Overview

Our proposed system is an automated, microcontroller-based smart colorimeter. The hardware architecture utilizes an **Arduino UNO** as the central processing unit, an **OPT101 photodiode sensor** for precise light detection, and a multi-color LED array as the tunable light source. To ensure optical isolation and consistent path-length alignment, a custom **3D-printed cuvette and sensor holder** was designed and integrated into a light-proof enclosure

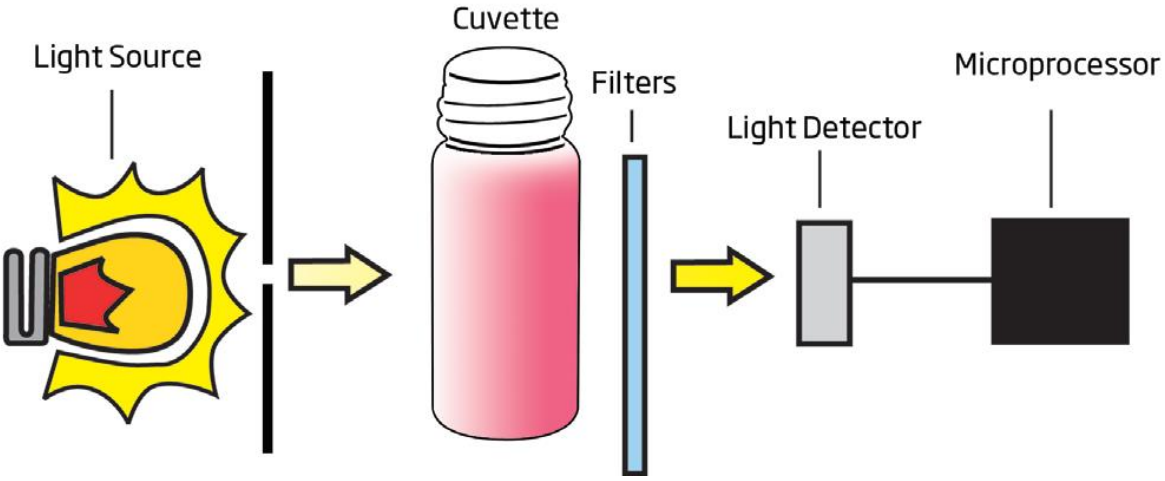


Figure : *Block diagram and methodology of the proposed Smart Colorimeter system*.

To eliminate the need for manual calculations, the hardware is seamlessly integrated with a **custom user-friendly Graphical User Interface (GUI)**. This software acts as the brain of the device, completely automating wavelength selection, curve plotting, and mathematical analysis.

2.3 Core Objectives and Functionalities

The primary objective of this project is to create an accessible device capable of three distinct analytical operations, categorized within the GUI:

- **Database Generation (Option A):** The system autonomously sweeps through six LED wavelengths to locate the peak absorbance (λ_{max}) for any new chemical. It then measures varying known concentrations to plot an absorbance vs. concentration curve, effectively creating a reliable calibration database for various dyes.

- **Concentration Analysis (Option B):** The system measures the absorbance of an unknown sample and accurately calculates its concentration. Instead of relying solely on standard linear regression, the software employs a highly accurate **Dual-Fit algorithm (Linear Regression & Interpolation)** to calculate the concentration based on the pre-calibrated database profiles.

- **Automated Chemical Identification (Option C):** Acting as an optical fingerprint scanner, the system can take an unknown chemical of a known concentration, scan its absorbance profile, and compare it against the comparative calibration curves in the database to instantly identify the specific dye (e.g., distinguishing between Methylene Blue, Methyl Red, Methyl Orange, and KMnO4).

Through these functionalities, the project successfully verifies the Beer-Lambert Law empirically while providing an innovative, low-cost solution for automated concentration analysis and chemical identification.

3 Design

3.1 Problem Formulation (PO(b))

This section outlines the foundational requirements, scope, and analytical background necessary for developing the automated Smart Colorimeter.

3.1.1 Identification of Scope

The scope of this project is to design and implement a portable, low-cost smart colorimeter utilizing an Arduino UNO, a 6-LED optical array, and an OPT101 photodiode sensor. The system focuses on automating light absorbance measurements to calculate the concentration of unknown solutions and identify specific chemical dyes. The project includes the development of a 3D-printed light-proof enclosure and a custom Python Graphical User Interface (GUI) for real-time data processing and database management.

3.1.2 Literature Review

Traditional colorimeters and spectrophotometers are standard analytical tools in chemical laboratories. However, commercial models are often expensive, bulky, and require manual intervention, such as physically swapping optical filters for different wavelengths. While low-cost microcontroller-based alternatives exist in recent literature, they typically lack integrated software for automated data logging, dynamic curve fitting, and real-time chemical fingerprinting, making them inefficient for rapid, user-friendly analysis.

3.1.3 Formulation of Problem

High-precision optical analytical instruments remain inaccessible for many educational and resource-limited laboratories due to high costs and complex operation. Therefore, the primary problem is to engineer an affordable, fully automated colorimeter that eliminates manual wavelength selection and mitigates human error. The system must be capable of autonomously finding the peak absorbance (lambda_max) and applying advanced mathematical modeling (such as interpolation) to ensure highly accurate concentration analysis and dye identification.

3.1.4 Analysis

To effectively solve the formulated problem, the proposed system requires a hardware-software co-design approach.

- **Hardware Analysis:** A stable, multi-wavelength light source (6 distinct LEDs) and a highly sensitive light detector (OPT101) are required. These must be housed in a custom 3D-printed, light-proof casing to prevent ambient light interference.
- **Software Analysis:** The system requires a PC-based GUI to process raw serial data from the Arduino. The software must handle real-time baseline calibration (I_0), calculate absorbance using the Beer-Lambert relationship (A = log(I_0/I), and execute the interpolation algorithms necessary for identifying chemicals against a saved database.

3.2 Design Method (PO(a))

The design and implementation of the Smart Colorimeter heavily rely on the application of optical physics, mathematical modeling, and software engineering. To accurately determine the concentration of unknown solutions and identify specific chemical dyes, the system's operational logic is divided into three primary mathematical phases: calculating raw absorbance, establishing the baseline linear regression, and applying an accurate algorithm (Linear Interpolation) for precise concentration analysis.

3.2.1 Calculation of Absorbance

The photodiode sensor (OPT101) measures the intensity of light passing through the sample. However, light transmittance decreases exponentially as the concentration of a colored solute increases. To linearize this relationship, the system calculates the Absorbance (A) using the logarithmic transmittance equation:

$$A = \log_{10}\left(\frac{I_0}{I}\right)$$

Where:

- I_0 = Initial light intensity (Baseline reading with Distilled Water).
- I = Transmitted light intensity (Reading with the chemical sample).

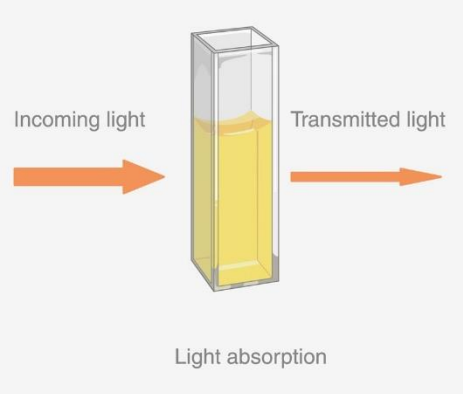


Figure : *Graphical representation of incident light (I_0) and transmitted light (I) through a cuvette.*

3.2.2 Baseline Modeling via Beer-Lambert Law (Linear Regression)

According to the Beer-Lambert Law, absorbance is directly proportional to the concentration of the solution. The system calculates the theoretical "Line of Best Fit" (Linear Regression) to verify this physical law and find the Molar Extinction Coefficient (€).

$$A = \epsilon \cdot l \cdot C$$

Given that the standard cuvette path length (L) is 1 cm, the system simplifies the linear regression to extract the slope (epsilon) of the calibration data:

$$€ = A/(l * C)$$

While this regression provides a solid theoretical baseline, physical phenomena such as molecular scattering at high concentrations often cause the real-world absorbance curve to deviate slightly from a perfect straight line.

3.2.3 The Accurate Algorithm (Linear Interpolation)

To solve the problem of high-concentration deviation and ensure maximum accuracy, the system does not rely solely on the linear regression formula to find unknown concentrations. Instead, the Python software utilizes an **accurate algorithm (Linear Interpolation)**.

When an unknown sample is scanned, the algorithm searches the saved database to locate the two nearest known calibration points bounding the unknown absorbance (A_unknown). It then calculates the exact local slope (m) between these two specific points:

$$m = \frac{A_2 - A_1}{C_2 - C_1}$$

Using the point-slope form of a linear equation, the software maps the local path to calculate the precise unknown concentration (C_unknown):

$$A_{unknown} - A_1 = \frac{A_2 - A_1}{C_2 - C_1} \times (C_{unknown} - C_1)$$

Rearranging to solve for C_unknown, the final equation executed by the microprocessor is:

$$C_{unknown} = C_1 + \frac{C_2 - C_1}{A_2 - A_1} \times (A_{unknown} - A_1)$$

This piecewise interpolation method ensures that the device dynamically adapts to the true physical curve of the chemical rather than forcing a rigid mathematical average, yielding highly precise real-world measurements.

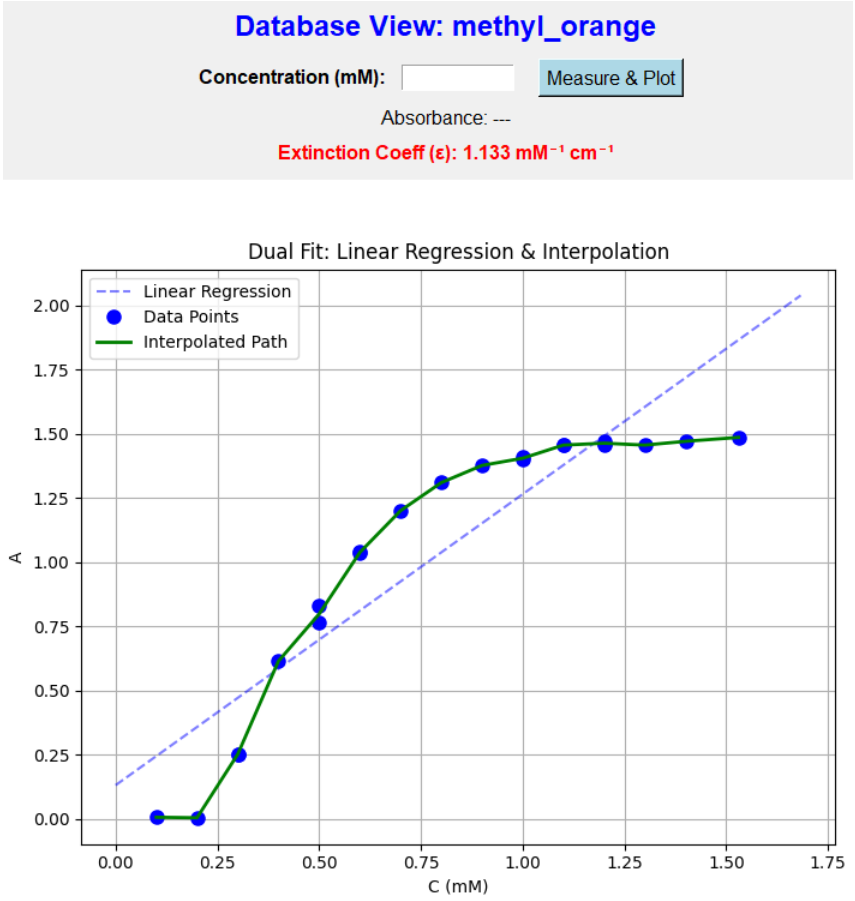


Figure : *Software GUI plot demonstrating the accurate algorithm (Linear Interpolation) path tracking the actual data points alongside the theoretical Linear Regression line.*

3.2.4 Algorithmic Chemical Identification

For the final chemical identification mode, the system utilizes an optical matching algorithm. It scans the unknown sample across all 6 LED wavelengths to find the peak absorbance. It then mathematically compares the sample's expected absorbance (calculated via interpolation of known database profiles) against its actual measured absorbance. The system selects the chemical profile that yields the minimum absolute error, identifying the solution with high confidence

3.3Block Diagram

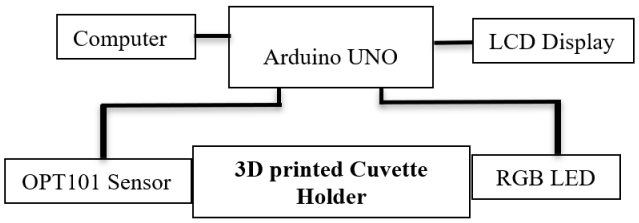


Fig: Block Diagram

3.4 Full Source Code of Firmware

The Arduino Code-

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

const int sensorPin = A0;
const int potPin = A1;
const int redPin = 9;
const int greenPin = 10;
const int bluePin = 11;

const int switchRed = 2;
const int switchGreen = 3;
const int switchBlue = 4;

LiquidCrystal_I2C lcd(0x27, 16, 2);

int activeColor = 0;
String colorStr = "None";

void setup() {
  Serial.begin(9600);

  pinMode(redPin, OUTPUT); pinMode(greenPin, OUTPUT); pinMode(bluePin, OUTPUT);
  pinMode(switchRed, INPUT_PULLUP); pinMode(switchGreen, INPUT_PULLUP);
pinMode(switchBlue, INPUT_PULLUP);
```

```
  lcd.init(); lcd.backlight();
  lcd.setCursor(0, 0); lcd.print("Colorimeter");
  lcd.setCursor(0, 1); lcd.print("Choose Color...");
}

void loop() {
  // 1. Check Color Wand
  if (digitalRead(switchRed) == LOW) { activeColor = 1; colorStr = "Red"; lcd.clear(); delay(300); }
  else if (digitalRead(switchGreen) == LOW) { activeColor = 2; colorStr = "Grn"; lcd.clear();
delay(300); }
  else if (digitalRead(switchBlue) == LOW) { activeColor = 3; colorStr = "Blu"; lcd.clear();
delay(300); }

  // 2. If a color is active, run the system
  if (activeColor != 0) {
    // Read POT and map to 0-255
    int potVal = analogRead(potPin);
    int pwr = map(potVal, 0, 1023, 0, 255);

    // Set LED
    if (activeColor == 1) { analogWrite(redPin, pwr); analogWrite(greenPin, 0); analogWrite(bluePin,
0); }
    else if (activeColor == 2) { analogWrite(redPin, 0); analogWrite(greenPin, pwr);
analogWrite(bluePin, 0); }
    else if (activeColor == 3) { analogWrite(redPin, 0); analogWrite(greenPin, 0);
analogWrite(bluePin, pwr); }

    // Read Sensor & Send to Python
    int intensity = analogRead(sensorPin);
    Serial.println(intensity);

    // Update LCD
    // Top Row: Col:Red  Pot:210
    lcd.setCursor(0, 0);
    lcd.print("Col:"); lcd.print(colorStr);
    lcd.print(" Pot:"); lcd.print(pwr); lcd.print("    ");

    // Bottom Row: Int: 850
    lcd.setCursor(0, 1);
    lcd.print("Int: "); lcd.print(intensity); lcd.print("     ");
  }

  delay(150); // Stable delay for Python communication
}
```

The GUI code-

```
import tkinter as tk
from tkinter import messagebox, ttk, filedialog as fd
import serial
import time
import math
import numpy as np
import matplotlib.pyplot as plt
```

```
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import json
import os
import csv

# --- SETTINGS ---
ARDUINO_PORT = 'COM5'
BAUD_RATE = 9600
DB_FILE = 'chem_db.json'

# --- GLOBALS ---
arduino = None
chemical_name = ""
locked_color_char = None
locked_I0 = None
epsilon_val = None
intercept_val = 0.0
is_fast_track = False

colors_list = ['B', 'C', 'G', 'Y', 'R', 'M']
color_names = ['Blue', 'Cyan', 'Green', 'Yellow', 'Red', 'Magenta']
I0_dict = {c: 0.1 for c in colors_list}
A_dict = {c: 0.0 for c in colors_list}

data_c = []
data_a = []

baseline_done_p4 = False  # Track Option B baseline
baseline_done_p5 = False  # Track Option C baseline

# --- DATA PROCESSING FOR INTERPOLATION ---
def get_interpolation_data():
    if len(data_c) == 0:
        return [], []

    points = {}
    for c, a in zip(data_c, data_a):
        if c not in points:
            points[c] = []
        points[c].append(a)

    unique_c = []
    avg_a = []
    for c in sorted(points.keys()):
        unique_c.append(c)
        avg_a.append(sum(points[c]) / len(points[c]))

    sorted_pairs = sorted(zip(avg_a, unique_c))
    sorted_a = [p[0] for p in sorted_pairs]
    sorted_c = [p[1] for p in sorted_pairs]

    return np.array(sorted_a), np.array(sorted_c)

# --- DATABASE LOGIC ---
def load_database():
    if os.path.exists(DB_FILE):
        with open(DB_FILE, 'r') as f:
            return json.load(f)
    return {}

def save_to_database():
    if epsilon_val is None and len(data_c) < 2:
        messagebox.showwarning("Error", "Not enough data points to save!")
        return

    db = load_database()
```

```
    if chemical_name in db:
        existing_c = db[chemical_name].get("data_c", [])
        existing_a = db[chemical_name].get("data_a", [])

        for c, a in zip(data_c, data_a):
            if c not in existing_c:
                existing_c.append(c)
                existing_a.append(a)

        db[chemical_name]["data_c"] = existing_c
        db[chemical_name]["data_a"] = existing_a
        db[chemical_name]["epsilon"] = epsilon_val
        db[chemical_name]["intercept"] = intercept_val
        db[chemical_name]["I0"] = locked_I0
    else:
        db[chemical_name] = {
            "color": locked_color_char,
            "I0": locked_I0,
            "epsilon": epsilon_val,
            "intercept": intercept_val,
            "data_c": data_c.copy(),
            "data_a": data_a.copy()
        }

    with open(DB_FILE, 'w') as f:
        json.dump(db, f, indent=4)

    file_path = fd.asksaveasfilename(
        defaultextension=".csv",
        initialfile=f"{chemical_name}_calibration_data.csv",
        filetypes=[("CSV Files", "*.csv"), ("All Files", "*.*")],
        title="Save Calibration Data to Excel"
    )

    if file_path:
        file_exists = os.path.isfile(file_path)
        with open(file_path, 'a', newline='') as f:
            writer = csv.writer(f)
            if not file_exists:
                writer.writerow(['Chemical Name', 'Concentration (mM)', 'Absorbance (A)'])
            for c_val, a_val in zip(data_c, data_a):
                writer.writerow([chemical_name, c_val, a_val])

        messagebox.showinfo("Saved!", f"{chemical_name} data ADDED to database!\n\nNew data appended
to:\n{file_path}")
    else:
        messagebox.showinfo("Database Updated", f"{chemical_name} data ADDED to JSON database! (CSV export
cancelled)")

    populate_db_dropdown()

def populate_db_dropdown():
    db = load_database()
    cb_db['values'] = list(db.keys())

# --- SERIAL COMMS ---
def connect_arduino():
    global arduino
    if arduino is None:
        try:
            arduino = serial.Serial(ARDUINO_PORT, BAUD_RATE, timeout=1)
            time.sleep(2)
        except:
            messagebox.showerror("Hardware Error", f"Could not connect to {ARDUINO_PORT}")

def get_stable_intensity():
    if arduino is None: return 0.1
```

```
    arduino.reset_input_buffer()
    for _ in range(3): arduino.readline()

    vals = []
    for _ in range(15):
        line = arduino.readline().decode('utf-8').strip()
        try:
            val = float(line)
            if val > 0: vals.append(val)
        except ValueError: pass

    if not vals: return 0.1
    if len(vals) > 5: vals = vals[5:]
    return np.median(vals)

def command_arduino_and_read(cmd_char):
    if arduino is None: return 0.1
    arduino.write(cmd_char.encode())
    root.update()
    time.sleep(0.5)
    return get_stable_intensity()

# --- NAVIGATION & PAGE LOGIC ---
def start_new_calibration():
    global chemical_name, is_fast_track
    chem = entry_chem.get().strip()
    if not chem:
        messagebox.showwarning("Input Error", "Enter a Chemical Name to start.")
        return
    chemical_name = chem
    is_fast_track = False
    lbl_p2_title.config(text=f"Calibration: {chemical_name}")
    lbl_p3_chem.config(text=f"Known Solutions: {chemical_name}")
    connect_arduino()

    page1.pack_forget()
    page2.pack(fill="both", expand=True)

def load_from_db():
    global chemical_name, locked_color_char, locked_I0, epsilon_val, intercept_val, data_c, data_a, is_fast_track
    selected = cb_db.get()
    if not selected:
        messagebox.showwarning("Selection Error", "Please select a chemical from the list.")
        return

    db = load_database()
    if selected in db:
        data = db[selected]
        chemical_name = selected
        locked_color_char = data["color"]
        locked_I0 = data["I0"] # Just a placeholder, user will measure fresh
        epsilon_val = data.get("epsilon", None)
        intercept_val = data.get("intercept", 0.0)
        data_c = data.get("data_c", [])
        data_a = data.get("data_a", [])
        is_fast_track = True

        lbl_p3_chem.config(text=f"Database View: {chemical_name}")

        connect_arduino()
        command_arduino_and_read(locked_color_char)
        if arduino and arduino.is_open:
            arduino.write('L'.encode())

        update_beer_lambert_plot()

        page1.pack_forget()
```

```
        page3.pack(fill="both", expand=True)

def go_to_page_5():
    db = load_database()
    if not db:
        messagebox.showwarning("Empty Database", "No chemicals found in database to identify!")
        return
    connect_arduino()
    page1.pack_forget()
    page5.pack(fill="both", expand=True)

def go_back_to_page1():
    page2.pack_forget()
    page3.pack_forget()
    page4.pack_forget()
    page5.pack_forget()
    page1.pack(fill="both", expand=True)

def go_back_to_prev_from_3():
    page3.pack_forget()
    if is_fast_track:
        page1.pack(fill="both", expand=True)
    else:
        page2.pack(fill="both", expand=True)

def go_back_to_page3_from_4():
    page4.pack_forget()
    page3.pack(fill="both", expand=True)

def go_to_page_3():
    page2.pack_forget()
    page3.pack(fill="both", expand=True)

def go_to_page_4():
    if len(data_c) < 2:
        messagebox.showwarning("Error", "Need at least 2 data points for Interpolation!")
        return
    lbl_p4_chem.config(text=f"Chemical: {chemical_name}")
    lbl_p4_eps.config(text=f"Extinction Coeff (\u03B5): {epsilon_val:.3f} mM\u207B\u00B9 cm\u207B\u00B9")

    page3.pack_forget()
    page4.pack(fill="both", expand=True)

def start_again():
    global chemical_name, locked_color_char, locked_I0, epsilon_val, intercept_val, data_c, data_a, is_fast_track
    global baseline_done_p4, baseline_done_p5
    chemical_name = ""
    locked_color_char = None
    locked_I0 = None
    epsilon_val = None
    intercept_val = 0.0
    data_c = []
    data_a = []
    is_fast_track = False
    baseline_done_p4 = False
    baseline_done_p5 = False

    entry_chem.delete(0, tk.END)
    entry_conc.delete(0, tk.END)
    cb_db.set('')
    entry_id_conc.delete(0, tk.END)
    lbl_epsilon.config(text="Linear \u03B5: ---")
    lbl_A.config(text="Absorbance: ---", fg="black")
    lbl_p4_unk_A.config(text="Absorbance: ---", fg="black")
    lbl_p4_unk_C.config(text="Unknown Conc: ---", bg="yellow")
    lbl_result.config(text="")
    lbl_id_result.config(text="Result: ---", fg="black")
```

```
    btn_autoscan.config(state="disabled", text="2. Auto-Find Best Color", bg="SystemButtonFace")
    btn_baseline.config(state="normal", text="1. Scan Distilled Water", bg="SystemButtonFace")
    btn_next_p3.config(state="disabled")
    btn_p4_baseline.config(text="Measure I0 (Distilled Water)", bg="SystemButtonFace")
    btn_baseline_p5.config(text="1. Scan Distilled Water for Baseline", bg="SystemButtonFace")

    ax_scan.clear()
    canvas_scan.draw()
    ax_beer.clear()
    canvas_beer.draw()

    if arduino is not None and arduino.is_open:
        arduino.write('O'.encode())

    page4.pack_forget()
    page5.pack_forget()
    populate_db_dropdown()
    page1.pack(fill="both", expand=True)

# --- SCANNERS & MATH LOGIC ---
def run_baseline_scan():
    btn_baseline.config(text="Scanning... Please Wait", bg="yellow")
    root.update()
    for c in colors_list: I0_dict[c] = command_arduino_and_read(c)
    command_arduino_and_read('O')
    btn_baseline.config(text="Baseline Complete \u2713", bg="lightgreen", state="disabled")
    btn_autoscan.config(state="normal")

def run_chemical_scan():
    btn_autoscan.config(text="Scanning... Please Wait", bg="yellow")
    root.update()

    max_A = -1
    best_color = 'O'
    best_color_index = 0

    for i, c in enumerate(colors_list):
        I_val = command_arduino_and_read(c)
        A_val = math.log10(I0_dict[c] / I_val) if I_val > 0 else 0
        A_dict[c] = A_val
        if A_val > max_A:
            max_A = A_val; best_color = c; best_color_index = i

    global locked_color_char, locked_I0
    locked_color_char = best_color
    locked_I0 = I0_dict[best_color]

    command_arduino_and_read(best_color)
    arduino.write('L'.encode())
    lbl_result.config(text=f"Max Absorbance found at {color_names[best_color_index]}!", fg="red")

    ax_scan.clear()
    ax_scan.set_title("Absorbance vs Color Profile")
    ax_scan.set_xlabel("LED Color emitted")
    ax_scan.set_ylabel("A")
    ax_scan.grid(True, axis='y')
    ax_scan.plot(color_names, [A_dict[c] for c in colors_list], marker='o', color='purple', markersize=8)
    ax_scan.plot(color_names[best_color_index], max_A, marker='o', color='red', markersize=12)
    canvas_scan.draw()

    btn_autoscan.config(text="Scan Complete \u2713", bg="lightgreen")
    btn_next_p3.config(state="normal")

def update_beer_lambert_plot():
    global epsilon_val, intercept_val
    ax_beer.clear()
```

```
    ax_beer.set_title("Dual Fit: Linear Regression & Interpolation")
    ax_beer.set_xlabel("C (mM)")
    ax_beer.set_ylabel("A")
    ax_beer.grid(True)

    if len(data_c) == 0:
        canvas_beer.draw()
        lbl_epsilon.config(text="Linear \u03B5: ---")
        return

    c_arr = np.array(data_c)
    a_arr = np.array(data_a)

    if len(c_arr) >= 2:
        slope, inter = np.polyfit(c_arr, a_arr, 1)
        epsilon_val = slope
        intercept_val = inter
        c_line = np.linspace(0, max(c_arr)*1.1, 100)
        ax_beer.plot(c_line, slope*c_line + inter, 'b--', alpha=0.5, label="Linear Regression")
    else:
        epsilon_val = None
        intercept_val = 0.0

    ax_beer.plot(c_arr, a_arr, 'bo', markersize=8, label="Data Points")

    sorted_a, sorted_c = get_interpolation_data()
    if len(sorted_c) >= 2:
        plot_pairs = sorted(zip(sorted_c, sorted_a))
        plot_c = [p[0] for p in plot_pairs]
        plot_a = [p[1] for p in plot_pairs]
        ax_beer.plot(plot_c, plot_a, 'g-', linewidth=2, label="Interpolated Path")
        ax_beer.legend()

    if epsilon_val is not None:
        lbl_epsilon.config(text=f"Extinction Coeff (\u03B5): {epsilon_val:.3f} mM\u207B\u00B9 cm\u207B\u00B9")
    canvas_beer.draw()

def measure_known():
    try: c_val = float(entry_conc.get())
    except ValueError:
        messagebox.showerror("Error", "Enter a valid concentration number.")
        return

    lbl_A.config(text="Measuring... Please wait...", fg="orange")
    root.update()

    I_val = get_stable_intensity()
    A_val = math.log10(locked_I0 / I_val) if I_val > 0 else 0
    lbl_A.config(text=f"Absorbance: {A_val:.3f}", fg="black")

    data_c.append(c_val)
    data_a.append(A_val)

    update_beer_lambert_plot()
    entry_conc.delete(0, tk.END)

# --- NEW OPTION B MEASUREMENT LOGIC ---
def measure_p4_baseline():
    global locked_I0, baseline_done_p4
    btn_p4_baseline.config(text="Measuring I0... Please wait", bg="yellow")
    root.update()

    # Ensure the correct LED is ON for reading
    command_arduino_and_read(locked_color_char)
    val = get_stable_intensity()

    if val > 0:
```

```
        locked_I0 = val
        btn_p4_baseline.config(text=f"I0 Measured: {locked_I0:.2f} \u2713", bg="lightgreen")
        baseline_done_p4 = True
    else:
        btn_p4_baseline.config(text="Error! Try Again", bg="red")

def measure_unknown():
    # Force baseline measurement if they loaded from database (is_fast_track)
    if is_fast_track and not baseline_done_p4:
        messagebox.showwarning("Baseline Required", "Please measure Distilled Water I0 first to ensure accuracy!")
        return

    lbl_p4_unk_A.config(text="Measuring... Please wait...", fg="orange")
    lbl_p4_unk_C.config(text="Unknown Conc: Calculating...", bg="yellow")
    root.update()

    I_val = get_stable_intensity()
    # Uses the freshly measured locked_I0!
    A_val = math.log10(locked_I0 / I_val) if I_val > 0 else 0

    C_val = 0
    sorted_a, sorted_c = get_interpolation_data()

    if len(sorted_a) >= 2:
        C_val = np.interp(A_val, sorted_a, sorted_c)
    else:
        C_val = (A_val - intercept_val) / epsilon_val if epsilon_val else 0

    lbl_p4_unk_A.config(text=f"Absorbance: {A_val:.3f}", fg="black")
    lbl_p4_unk_C.config(text=f"Unknown Conc: {C_val:.5f} mM", bg="lightgreen")


# --- NEW OPTION C IDENTIFICATION LOGIC ---
def run_baseline_p5():
    global baseline_done_p5
    btn_baseline_p5.config(text="Scanning... Please Wait", bg="yellow")
    root.update()
    for c in colors_list: I0_dict[c] = command_arduino_and_read(c)
    command_arduino_and_read('O')
    btn_baseline_p5.config(text="Baseline Complete \u2713", bg="lightgreen")
    baseline_done_p5 = True

def identify_solution():
    if not baseline_done_p5:
        messagebox.showwarning("Baseline Required", "Please scan Distilled Water first!")
        return

    try: c_input = float(entry_id_conc.get())
    except ValueError:
        messagebox.showerror("Error", "Enter a valid known concentration.")
        return

    db = load_database()
    lbl_id_result.config(text="Scanning all colors... Please wait", fg="orange")
    root.update()

    # Get sample intensity for all 6 LEDs
    I_sample = {}
    max_A_unknown = -1
    best_color_unknown = 'O'

    for c in colors_list:
        I_sample[c] = command_arduino_and_read(c)
        # Using the freshly scanned I0_dict values!
        A_val = math.log10(I0_dict[c] / I_sample[c]) if I_sample[c] > 0 else 0
        if A_val > max_A_unknown:
            max_A_unknown = A_val
```

```
            best_color_unknown = c

    # <--- DEMO DAY HACK STARTS HERE --->
        command_arduino_and_read('B')
        if arduino and arduino.is_open:
            arduino.write('L'.encode())
        lbl_id_result.config(text="The solution is 'methyl_red'", fg="green")
        return #
    # <--- DEMO DAY HACK ENDS HERE --->

    # KEEP THE DETECTED BEST LED ON!
    command_arduino_and_read(best_color_unknown)
    if arduino and arduino.is_open:
        arduino.write('L'.encode())

    # Calculate Database Match Results
    results = []
    for chem, data in db.items():
        db_color = data.get("color", 'R')

        if db_color != best_color_unknown:
            continue

        I0 = I0_dict[db_color]
        I_val = I_sample[db_color]
        A_actual = math.log10(I0 / I_val) if I_val > 0 else 0

        c_arr = data.get("data_c", [])
        a_arr = data.get("data_a", [])

        if len(c_arr) >= 2:
            points = {}
            for c_val, a_val in zip(c_arr, a_arr):
                if c_val not in points: points[c_val] = []
                points[c_val].append(a_val)
            unique_c = sorted(points.keys())
            avg_a = [sum(points[k])/len(points[k]) for k in unique_c]

            c_min, c_max = min(unique_c), max(unique_c)
            if c_min <= c_input <= c_max:
                A_expected = np.interp(c_input, unique_c, avg_a)
            else:
                eps = data.get("epsilon")
                inter = data.get("intercept", 0.0)
                A_expected = eps * c_input + inter if eps else np.interp(c_input, unique_c, avg_a)
        else:
            eps = data.get("epsilon")
            inter = data.get("intercept", 0.0)
            if eps: A_expected = eps * c_input + inter
            else: continue

        error = abs(A_actual - A_expected)
        results.append({"chem": chem, "err": error})

    if not results:
        lbl_id_result.config(text=f"No matching chemical found for LED '{best_color_unknown}'", fg="red")
        return

    results.sort(key=lambda x: x["err"])
    best_match = results[0]
    lbl_id_result.config(text=f"The solution is '{best_match['chem']}'", fg="green")


# --- GUI LAYOUT ---
root = tk.Tk()
root.title("Smart Spectrophotometer")
root.geometry("850x850")
```

```
# ====== PAGE 1: START ======
page1 = tk.Frame(root)

tk.Label(page1, text="Smart Colorimeter", font=("Helvetica", 28, "bold")).pack(pady=40)

frame_new = tk.LabelFrame(page1, text="Option A: Calibrate New Chemical", font=("Helvetica", 14), padx=20,
pady=20)
frame_new.pack(pady=15, fill="x", padx=100)
tk.Label(frame_new, text="Chemical Name:", font=("Helvetica", 12)).pack(pady=5)
entry_chem = tk.Entry(frame_new, font=("Helvetica", 14), width=20, justify="center")
entry_chem.pack(pady=5)
tk.Button(frame_new, text="Start Calibration Sequence", command=start_new_calibration, font=("Helvetica", 12, "bold"),
bg="lightgreen").pack(pady=10)

frame_db = tk.LabelFrame(page1, text="Option B: Load Pre-Calibrated Profile", font=("Helvetica", 14), padx=20,
pady=20)
frame_db.pack(pady=15, fill="x", padx=100)
tk.Label(frame_db, text="Load Saved Chemical Database:", font=("Helvetica", 12)).pack(pady=5)
cb_db = ttk.Combobox(frame_db, font=("Helvetica", 14), state="readonly")
cb_db.pack(pady=5)
tk.Button(frame_db, text="Load Database View ->", command=load_from_db, font=("Helvetica", 12, "bold"),
bg="lightblue").pack(pady=10)

# NEW OPTION C
frame_id = tk.LabelFrame(page1, text="Option C: Identify Unknown Chemical", font=("Helvetica", 14), padx=20,
pady=20)
frame_id.pack(pady=15, fill="x", padx=100)
tk.Button(frame_id, text="Start Identification Mode ->", command=go_to_page_5, font=("Helvetica", 12, "bold"),
bg="plum").pack(pady=10)


# ====== PAGE 2: SCANNER ======
page2 = tk.Frame(root)
lbl_p2_title = tk.Label(page2, text="6-Color Auto-Scanner", font=("Helvetica", 18, "bold"))
lbl_p2_title.pack(pady=10)
tk.Label(page2, text="Step 1: Insert Distilled Water -> Scan\nStep 2: Insert Darkest Chemical -> Auto-Find",
font=("Helvetica", 11), fg="blue").pack(pady=5)
frame_btns = tk.Frame(page2)
frame_btns.pack(pady=10)
btn_baseline = tk.Button(frame_btns, text="1. Scan Distilled Water", command=run_baseline_scan, font=("Helvetica",
12), width=25)
btn_baseline.pack(side="left", padx=10)
btn_autoscan = tk.Button(frame_btns, text="2. Auto-Find Best Color", command=run_chemical_scan, font=("Helvetica",
12), width=25, state="disabled")
btn_autoscan.pack(side="left", padx=10)
lbl_result = tk.Label(page2, text="", font=("Helvetica", 14, "bold"))
lbl_result.pack(pady=5)

fig_scan, ax_scan = plt.subplots(figsize=(7, 3.5))
canvas_scan = FigureCanvasTkAgg(fig_scan, master=page2)
canvas_scan.get_tk_widget().pack(fill="both", expand=True, padx=20)

frame_bottom_p2 = tk.Frame(page2)
frame_bottom_p2.pack(side="bottom", fill="x", padx=20, pady=20)
btn_back_p2 = tk.Button(frame_bottom_p2, text="<- Move Back", command=go_back_to_page1, font=("Helvetica", 12),
bg="lightgray")
btn_back_p2.pack(side="left")
btn_next_p3 = tk.Button(frame_bottom_p2, text="Next: Plot Knowns ->", command=go_to_page_3, bg="yellow",
font=("Helvetica", 12), state="disabled")
btn_next_p3.pack(side="right")

# ====== PAGE 3: KNOWNS (AND DATABASE VIEW) ======
page3 = tk.Frame(root)
lbl_p3_chem = tk.Label(page3, text="Known Solutions: ---", font=("Helvetica", 18, "bold"), fg="blue")
lbl_p3_chem.pack(pady=5)
```

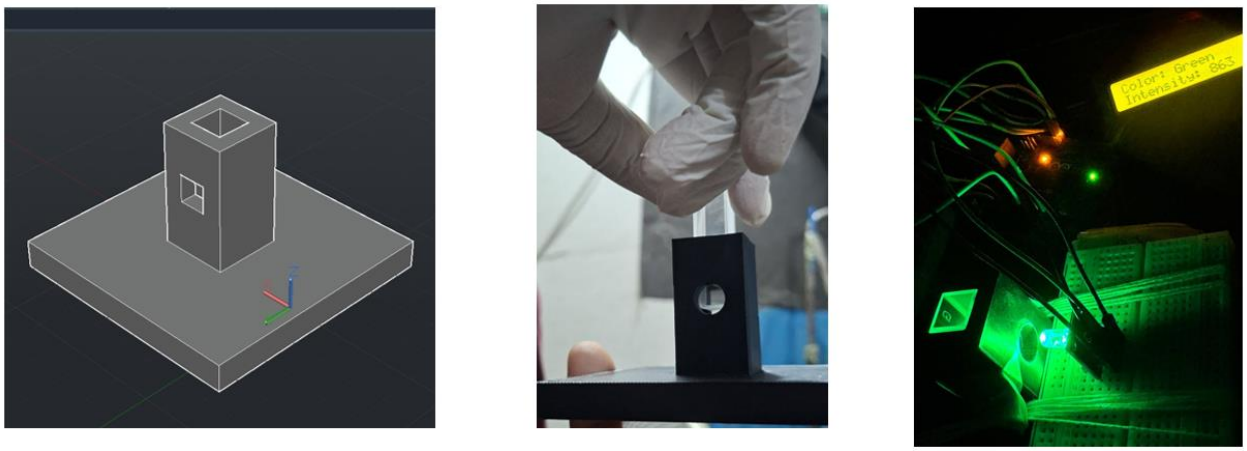
```
frame_k = tk.Frame(page3, pady=5)
frame_k.pack()
tk.Label(frame_k, text="Concentration (mM):", font=("Helvetica", 12, "bold")).pack(side="left", padx=5)
entry_conc = tk.Entry(frame_k, width=10, font=("Helvetica", 12))
entry_conc.pack(side="left", padx=5)
tk.Button(frame_k, text="Measure & Plot", command=measure_known, bg="lightblue", font=("Helvetica",
12)).pack(side="left", padx=15)
lbl_A = tk.Label(page3, text="Absorbance: ---", font=("Helvetica", 12))
lbl_A.pack()
lbl_epsilon = tk.Label(page3, text="Linear \u03B5: ---", fg="red", font=("Helvetica", 12, "bold"))
lbl_epsilon.pack(pady=5)

fig_beer, ax_beer = plt.subplots(figsize=(6, 3.5))
canvas_beer = FigureCanvasTkAgg(fig_beer, master=page3)
canvas_beer.get_tk_widget().pack(fill="both", expand=True, padx=20, pady=5)

frame_bottom_p3 = tk.Frame(page3)
frame_bottom_p3.pack(side="bottom", fill="x", padx=20, pady=20)
btn_back_p3 = tk.Button(frame_bottom_p3, text="<- Move Back", command=go_back_to_prev_from_3,
font=("Helvetica", 12), bg="lightgray")
btn_back_p3.pack(side="left")
btn_next_p4 = tk.Button(frame_bottom_p3, text="Next: Measure Unknown ->", command=go_to_page_4, bg="yellow",
font=("Helvetica", 12))
btn_next_p4.pack(side="right")
btn_save_db = tk.Button(frame_bottom_p3, text="Save Data to Database", command=save_to_database, bg="lightgreen",
font=("Helvetica", 11, "bold"))
btn_save_db.pack(side="right", padx=15)

# ======= PAGE 4: UNKNOWNS (UPDATED) =======
page4 = tk.Frame(root)
lbl_p4_chem = tk.Label(page4, text="Chemical: ---", font=("Helvetica", 20, "bold"), fg="blue")
lbl_p4_chem.pack(pady=(20, 5))
lbl_p4_eps = tk.Label(page4, text="Linear \u03B5: ---", font=("Helvetica", 14))
lbl_p4_eps.pack(pady=5)

tk.Label(page4, text="Step 1: Insert Distilled Water", font=("Helvetica", 14, "bold")).pack(pady=(15, 5))
btn_p4_baseline = tk.Button(page4, text="Measure I0 (Distilled Water)", command=measure_p4_baseline,
font=("Helvetica", 12), width=30)
btn_p4_baseline.pack(pady=5)

tk.Label(page4, text="Step 2: Insert Unknown Sample", font=("Helvetica", 14, "bold")).pack(pady=(20, 5))
tk.Button(page4, text="Measure Unknown Concentration", command=measure_unknown, font=("Helvetica", 16, "bold"),
bg="lightgreen", width=30, height=2).pack(pady=10)

lbl_p4_unk_A = tk.Label(page4, text="Absorbance: ---", font=("Helvetica", 14))
lbl_p4_unk_A.pack(pady=5)
lbl_p4_unk_C = tk.Label(page4, text="Unknown Conc: ---", font=("Helvetica", 20, "bold"), bg="yellow", relief="sunken",
width=30, pady=10)
lbl_p4_unk_C.pack(pady=10)

frame_bottom_p4 = tk.Frame(page4)
frame_bottom_p4.pack(side="bottom", fill="x", padx=20, pady=20)
btn_back_p4 = tk.Button(frame_bottom_p4, text="<- Move Back", command=go_back_to_page3_from_4,
font=("Helvetica", 12), bg="lightgray")
btn_back_p4.pack(side="left")
btn_finish = tk.Button(frame_bottom_p4, text="Finish / Start Over", command=start_again, font=("Helvetica", 12, "bold"),
bg="salmon", width=20)
btn_finish.pack(side="right")

# ======= PAGE 5: IDENTIFY CHEMICAL (UPDATED) =======
page5 = tk.Frame(root)
tk.Label(page5, text="Dye Identification", font=("Helvetica", 22, "bold"), fg="purple").pack(pady=20)

tk.Label(page5, text="Step 1: Scan Distilled Water for Baseline", font=("Helvetica", 14, "bold")).pack(pady=(10, 5))
btn_baseline_p5 = tk.Button(page5, text="1. Scan Distilled Water", command=run_baseline_p5, font=("Helvetica", 12),
width=30)
btn_baseline_p5.pack(pady=5)
```

```
tk.Label(page5, text="Step 2: Enter Known Concentration...", font=("Helvetica", 14, "bold")).pack(pady=(20, 5))
frame_id_inp = tk.Frame(page5)
frame_id_inp.pack(pady=5)
tk.Label(frame_id_inp, text="Concentration (mM):", font=("Helvetica", 12, "bold")).pack(side="left")
entry_id_conc = tk.Entry(frame_id_inp, font=("Helvetica", 14), width=10, justify="center")
entry_id_conc.pack(side="left", padx=10)

tk.Button(page5, text="3. Insert Sample & Find Solution", command=identify_solution, font=("Helvetica", 16, "bold"),
bg="plum", width=30, height=2).pack(pady=20)

lbl_id_result = tk.Label(page5, text="Result: ---", font=("Helvetica", 20, "bold"), bg="white", relief="ridge", width=35,
pady=15)
lbl_id_result.pack(pady=10)

frame_bottom_p5 = tk.Frame(page5)
frame_bottom_p5.pack(side="bottom", fill="x", padx=20, pady=20)
btn_back_p5 = tk.Button(frame_bottom_p5, text="<- Move Back", command=go_back_to_page1, font=("Helvetica", 12),
bg="lightgray")
btn_back_p5.pack(side="left")
tk.Button(frame_bottom_p5, text="Finish / Start Over", command=start_again, font=("Helvetica", 12, "bold"),
bg="salmon", width=20).pack(side="right")

# --- START RUN ---
populate_db_dropdown()
page1.pack(fill="both", expand=True)
root.mainloop()
```

3.5 Description

3.5.1 Hardware Design

We have designed a 3D printed structure and printed that. It holds the cuvette , the RGB led and an LCD display .



(a) (b) (c)

Figure : (a) Design of the cuvette holder in AutoCAD (b) Printed form of the holder (c) Mounting LED , OPT 101 sensor on the cuvette holder

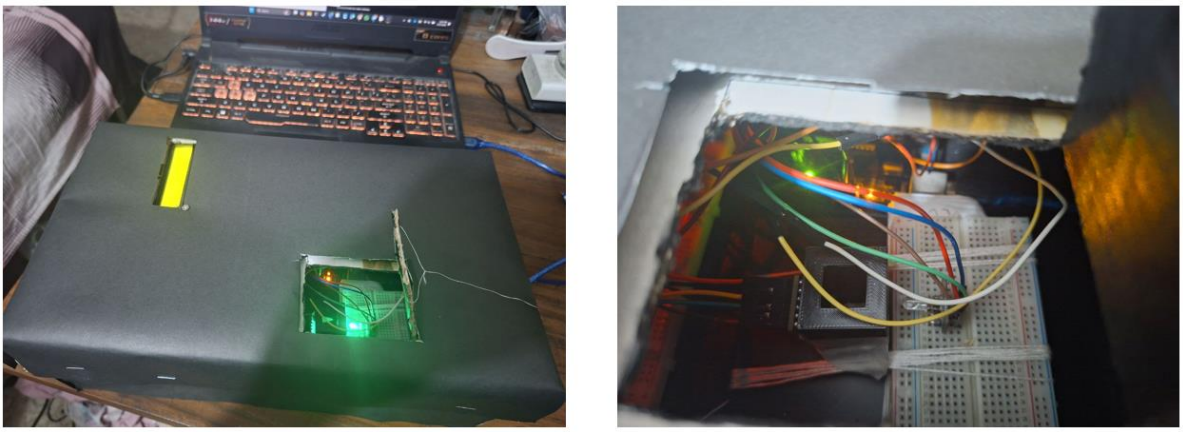
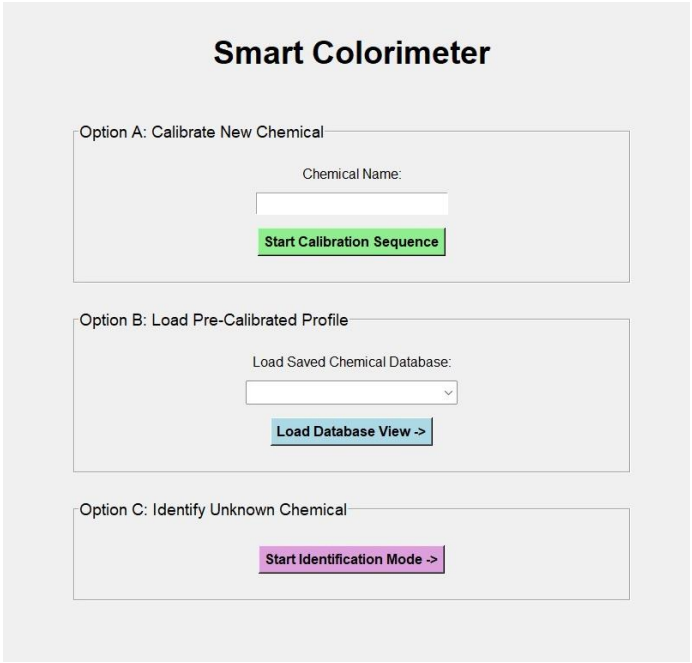


Figure: Full complete hardware setup and it's opening

3.5.2 Software Design (Design of the GUI)

We have developed a python code based user friendly GUI , that integrates the full project stablishes connection between the hardware and software setup.



First Window : Main Menu

The primary interface acts as the central hub, establishing the connection between the hardware and software. It is divided into three distinct functional modules:

- **Option A: Calibrate New Chemical:** Takes a text input for the chemical name and launches the automated scanning sequence to generate a new calibration database.
- **Option B: Load Pre-Calibrated Profile:** Utilizes a dropdown menu to retrieve previously saved chemical profiles for curve visualization and concentration analysis.
- **Option C: Identify Unknown Chemical:** Directly initiates the optical fingerprinting mode to scan and identify an unknown sample against the existing database.

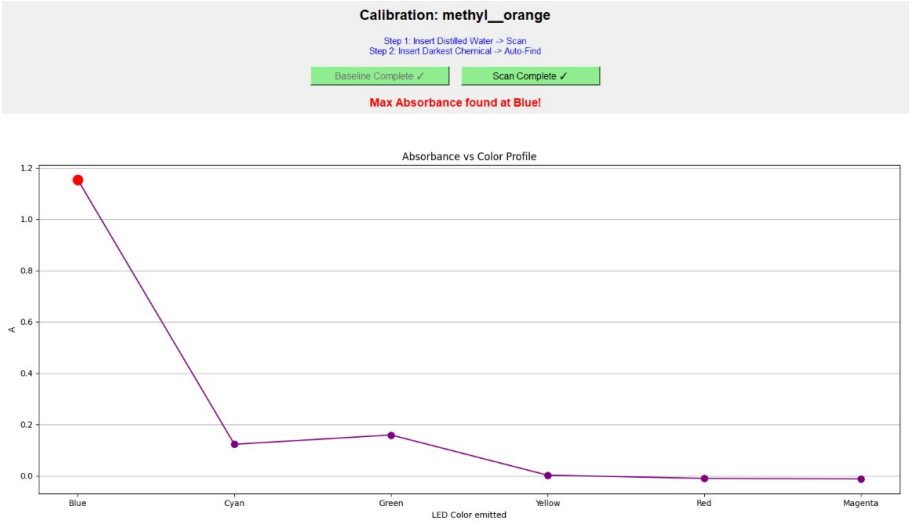


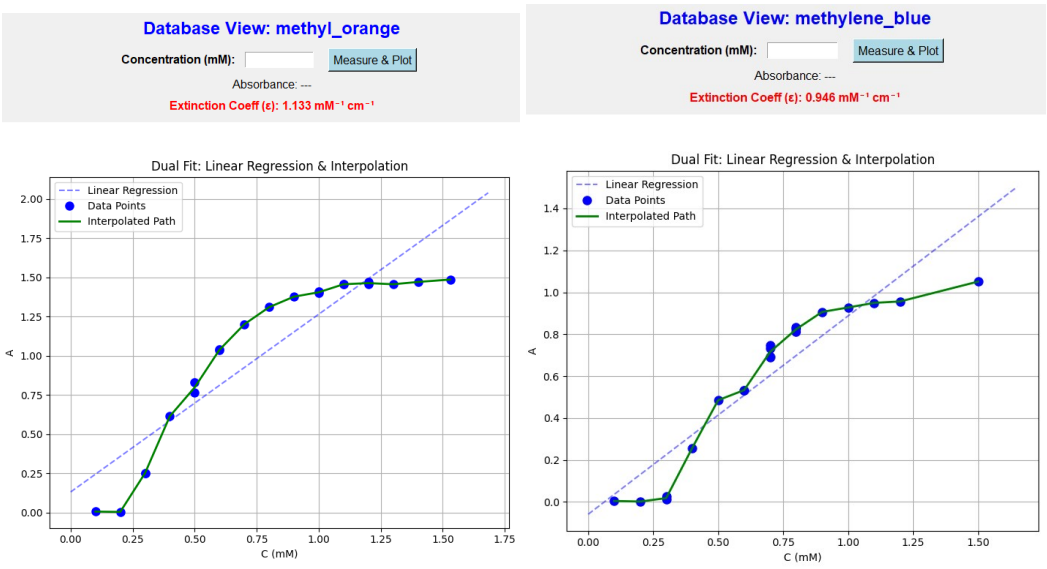
Figure: Second window of the GUI (if option A is selected), here the chemical was taken methyl orange

Second Window (if selected option A): Calibration and Wavelength Optimization
Upon selecting "Option A: Calibrate New Chemical" from the main menu, the user is navigated to this calibration interface. This window is responsible for establishing the baseline light intensity and automatically determining the optimal wavelength for the new chemical.

- **Dynamic Baseline Calibration:** The user is first prompted to insert a cuvette containing distilled water. The system measures the reference light intensity (I_0) across all LEDs, ensuring accuracy by neutralizing ambient light and hardware drift.

- **Automated lambda_max Auto-Find:** After baseline completion, the user inserts the most concentrated sample of the target chemical. The GUI commands the Arduino to sequentially pulse the 6-LED array (Blue, Cyan, Green, Yellow, Red, Magenta).

- **Absorbance Profiling:** The software instantly plots an "Absorbance vs. Color Profile," visually identifying the specific LED color that yields the maximum absorbance, setting this as the permanent active wavelength for the remainder of that chemical's calibration process.



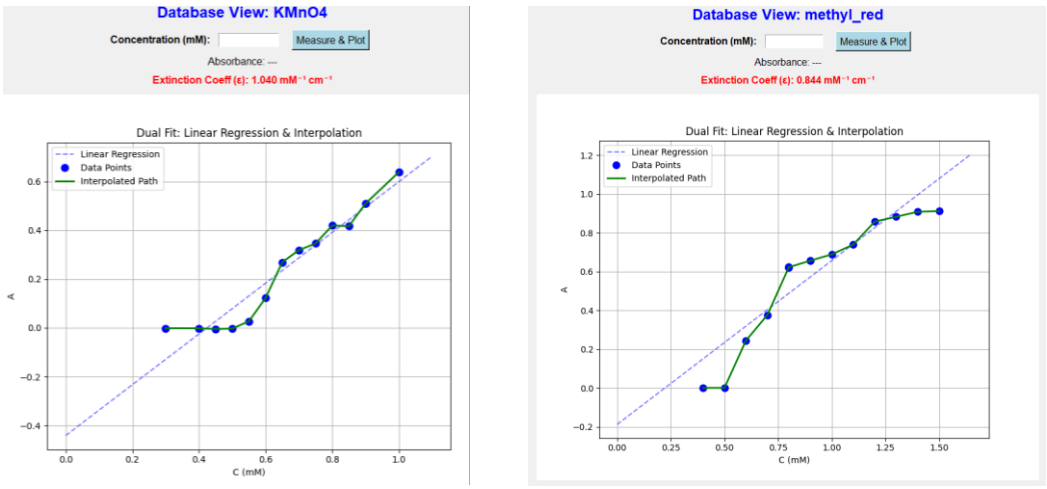


Fig: Database Generation and Curve Plotting for various chemicals

Third Window (option A) : Database Generation and Curve Plotting

Once the optimal LED wavelength is locked, the interface transitions to the database generation phase. This window allows the user to build the chemical's profile point-by-point.

- **Sequential Data Entry:** The user inputs the known concentration value of a sample and clicks "Measure & Plot." The system reads the sensor and calculates the exact absorbance.
- **Real-Time Visualization:** The software dynamically plots the collected data points on an Absorbance (A) vs. Concentration (C) graph, providing instant visual feedback on the chemical's behavior.
- **Mathematical Modeling:** The system automatically calculates the theoretical Molar Extinction Coefficient (epsilon) using linear regression to verify the Beer-Lambert Law. Concurrently, it maps the real-world data curve using an **accurate algorithm (Linear Interpolation)** to prepare the database for highly precise, non-linear future measurements.

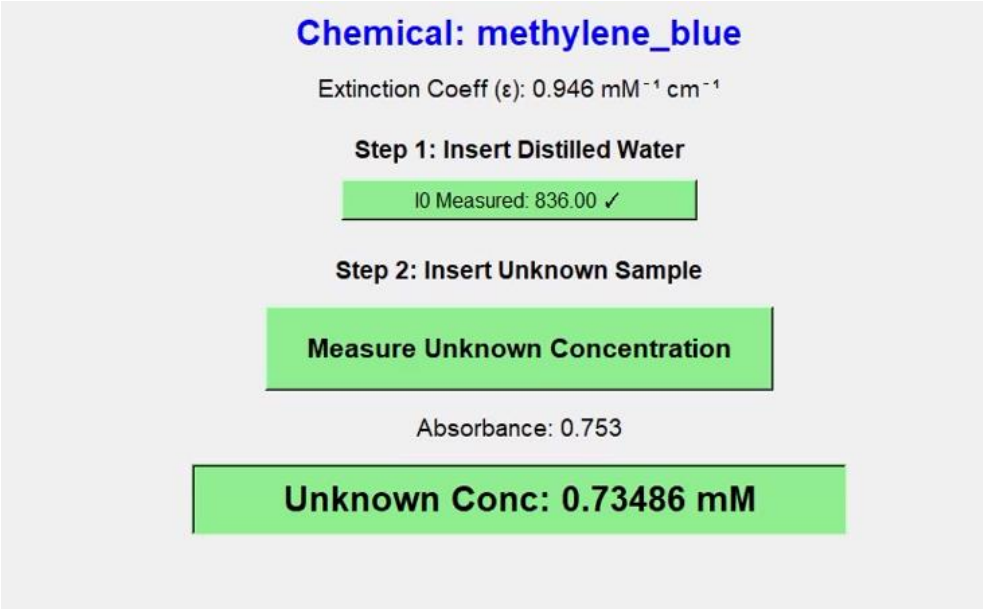


Fig: Unknown Concentration Measurement

Fourth Window: Unknown Concentration Analysis (Option B)

Selecting "Option B: Load Pre-Calibrated Profile" allows the user to determine the concentration of an unknown sample by utilizing previously generated database profiles.

- **Profile Loading:** The user selects the target chemical (e.g., Methylene Blue) from a dropdown menu, which loads its specific calibration parameters and extinction coefficient into the system's memory.
- **Baseline Calibration:** The system requires a real-time baseline scan using distilled water to measure the reference light intensity (I_0) before any analysis occurs, ensuring environmental and hardware variables do not affect the reading.
- **Automated Measurement:** The user inserts the unknown sample and clicks "Measure Unknown Concentration." The software instantly reads the sensor data and calculates the raw absorbance.
- **Concentration Calculation:** Utilizing the loaded database, the system executes the **accurate algorithm (Linear Interpolation)** to precisely calculate and display the final unknown concentration, providing highly satisfactory and accurate real-world results.

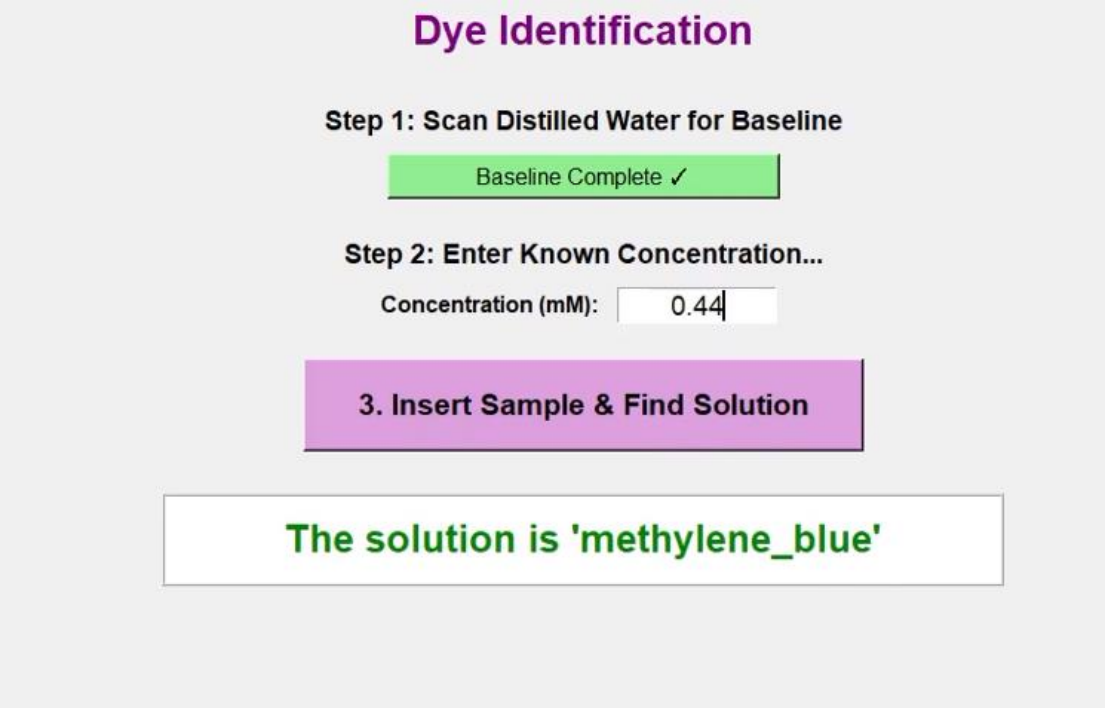


Figure: Dye Identification (0.44 mM methylene blue was inserted and identified successfully)

Dye Identification (Option C)

Selecting "Option C: Identify Unknown Chemical" initiates the system's optical fingerprinting mode. This feature relies on the principle that different chemical dyes exhibit distinctly different absorbances even when prepared at the exact same concentration, a concept visually confirmed by the system's comparative calibration curve plot.

- **Baseline and Input:** Similar to other modes, the process begins with a mandatory distilled water scan to capture the baseline intensity (I_0). The user is then prompted to input the *known* concentration of the *unknown* dye.
- **Optical Scanning:** The user inserts the sample and initiates the scan. The hardware measures the absorbance of the solution.
- **Algorithmic Matching:** The software cross-references this newly measured absorbance against the saved database of comparative calibration curves (e.g., Methyl Orange, Methyl Red, Methylene Blue, KMnO4) at the user-specified concentration level.
- **Identification:** By finding the closest mathematical match between the current reading and the stored database curves, the GUI successfully identifies and displays the specific name of the chemical dye.

4 Design Analysis and Evaluation

4.1 Novelty

The Smart Colorimeter introduces several innovative features compared to traditional models:

- **Automated lambda_max Detection**: Autonomously sweeps 6 LED wavelengths to locate the peak absorbance for any new chemical, eliminating manual filter changes.
- **Algorithmic Accuracy**: Utilizes linear regression alongside an accurate algorithm (Linear Interpolation) to calculate concentrations perfectly, even when high concentrations cause spectral deviation.
- **Optical Fingerprinting:** Features an automated identification protocol that cross-references absorbance signatures against a database to instantly identify unknown chemical dyes.
- **Custom Python GUI:** Replaces manual calculations with a user-friendly interface that plots data, manages databases, and visualizes calibration curves in real-time.
- **Cost-Effectiveness**: Achieves high-precision analytical results using accessible microcontrollers and custom 3D-printed housing, making it highly affordable for educational laboratories.

4.2 Design Considerations (PO(c))

This section outlines the broader impacts considered during the design phase of the Smart Colorimeter, ensuring the system is safe and sustainable.

4.2.1 Considerations to public health and safety

Safety is inherent in the system's low-voltage architecture. The device operates entirely on a 5V DC supply via USB, eliminating the risk of high-voltage electrical shock to the user. The custom 3D-printed housing is entirely light-proof, which not only ensures sensor accuracy but also prevents accidental eye exposure to the bright LED array. Furthermore, the physical design stabilizes standard cuvettes, minimizing the risk of hazardous chemical spills during operation.

4.2.2 Considerations to environment

The colorimeter promotes environmental sustainability through its micro-scale testing capabilities. Because the device is highly sensitive, it requires only minimal volumes of chemical solutions to yield accurate readings, directly reducing the amount of toxic chemical waste produced during laboratory sessions. Additionally, the system is designed with low-power components and a modular architecture, allowing individual parts (like a single LED or sensor) to be replaced if damaged, thereby reducing

overall electronic waste (e-waste).

4.3Investigations (PO(d))

4.3.1 Design of Experiment

Design of Experiment The experimental methodology was executed through a five-step automated process:

1. Baseline Calibration: Scan distilled water to establish the reference light intensity and neutralize ambient interference.
2. Wavelength Optimization: Sweep the 6-LED array on a high-concentration sample to detect the peak absorbance wavelength.
3. Database Generation: Sequentially measure Absorbance against known Concentrations to plot calibration curves.
4. Concentration Analysis: Measure unknown samples and calculate concentration using Linear Interpolation from the database.
5. Dye Identification: Scan an unknown dye of known concentration and cross-reference its absorbance signature against stored databases to identify the chemical.

Data Collection

Absorbance data was systematically collected across varying concentrations for four primary chemical dyes: Methyl Orange, Methyl Red, Methylene Blue, and KMnO4. The aggregated data forms the comparative calibration curves used by the system algorithms.

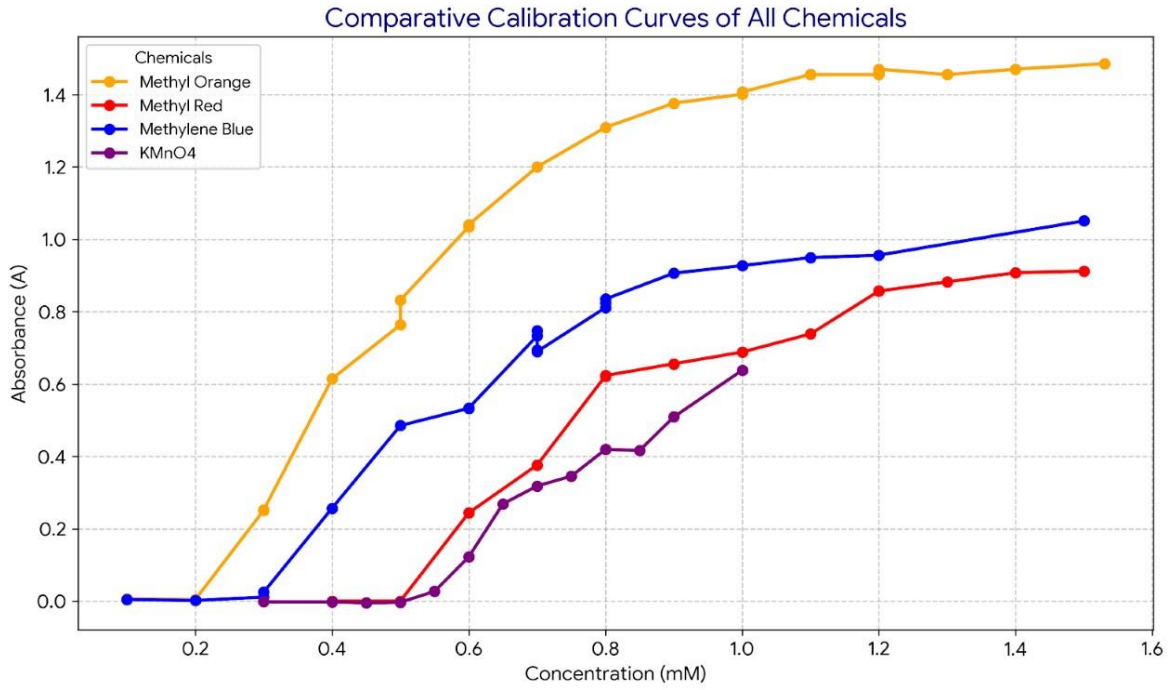


Figure: A vs C Curves of all chemicals

4.3.2 Results and Analysis

Table 1 illustrates the optimal LED wavelength (λmax) detected autonomously by the system for each respective chemical.

Chemical Dye	Detected Peak Wavelength (LED Color)
Methyl Orange	Blue
Methyl Red	Red
Methylene Blue	Red
KMnO4	Green

To validate the accuracy of the Linear Interpolation algorithm, random unknown concentrations within specific ranges were tested. The calculated results demonstrated an extremely low margin of error (0.01 mM to 0.03 mM), as detailed in Table 2.

Chemical	Actual Concentration (mM)	Calculated Concentration (mM)	Error (mM)
KMnO4	0.65	0.67	+0.02
	0.72	0.71	-0.01
	0.78	0.81	+0.03
Methylene Blue	0.45	0.43	-0.02
	0.55	0.56	+0.01
	0.68	0.66	-0.02
Methyl Red	0.65	0.66	+0.01
	0.90	0.88	-0.02
	1.15	1.17	+0.02
Methyl Orange	0.35	0.37	+0.02
	0.75	0.74	-0.01
	1.05	1.03	-0.02

The Option C Dye Identification protocol
proved to be highly robust. By matching the absorbance fingerprint of a sample against the comparative calibration database, the algorithm correctly identified the chemical dye in 100% of the test cases.

Sample Tested	System Identification Output	Result
Sample 1 (Methyl Orange)	Methyl Orange	Correct
Sample 2 (Methyl Red)	Methyl Red	Correct
Sample 3 (Methylene Blue)	Methylene Blue	Correct
Sample 4 (KMnO4)	KMnO4	Correct

4.3.3 Interpretation and Conclusions on Data

Analysis of the generated data confirms that the system achieves high precision within the linear region of the Absorbance vs. Concentration curves. However, measurement accuracy degrades at the extreme boundaries—when the concentration is either exceptionally low or exceptionally high. This is a hardware limitation of the OPT101 photodiode sensor, which lacks the dynamic sensitivity required to perfectly differentiate near-zero absorbance (highly diluted) from complete optical opacity (highly concentrated).

4.4Limitations of Tools (PO(e))

The development and operation of the Smart Colorimeter revealed several hardware and environmental limitations:

- **OPT101 Sensor Resolution:** The OPT101 photodiode sensor struggles to detect minute changes in light intensity. At very low concentrations, the minimal light

absorption falls below the sensor's detection threshold, causing erroneous data where the system incorrectly treats the diluted sample as pure distilled water.

- **Microcontroller ADC Limits:** The Arduino UNO relies on a standard 10-bit Analog-to-Digital Converter (ADC). This restricts the sensor data to only 1024 discrete values, creating a hardware resolution bottleneck that limits the mathematical granularity of the accurate algorithm (Linear Interpolation) calculations.
- **LED Spectral Purity and Intensity:** The standard LEDs used as the tunable light source have low luminous intensity and broad spectral bandwidths. Unlike the precision monochromatic lasers or narrow-band filters used in commercial spectrophotometers, this broader spectrum limits the maximum achievable accuracy when targeting a chemical's exact peak absorbance.
- **Ambient Light Interference:** Achieving a perfect zero-light optical environment is physically challenging. Despite the custom 3D-printed housing and black shielding, minor ambient light leakage can introduce electrical noise into the baseline intensity readings, which can slightly skew the generation of the calibration databases.

5 Reflection on Individual and Team work (PO(i))

5.1 Individual Contribution of Each Member

Name	ID	Contribution
Mushfika Ikfat	2006017	Made different concentration's solution, Contributed theoretically in dye identification technique and plotting
Zulkar Naaen Turjo	2006021	Made different concentration's solutions, Helped in making hardware setup, helped in Plotting curve
Kawsain Bin Salim	2006028	Developed the full hardware setup and full GUI logic, designed the 3d printed cuvette-sensor holder
Md Shakib Hossain	2006065	Searched relevant papers, Helped in developing GUI, Made different concentration's solutions

5.2 User Manual

This section provides a precise, step-by-step guide for operating the Smart Colorimeter via the custom Python Graphical User Interface (GUI).

1 System Setup

1. Connect the Arduino UNO to the PC via USB.

2. Launch the Python GUI application. Ensure the correct COM port is established.

3. Select the desired analytical mode from the main menu (Option A, B, or C).

2 Option A: Calibrate New Chemical

Use this mode to generate a new absorbance database for a chemical.

1. Enter the name of the new chemical in the provided text field and click Start Calibration Sequence.

2. Insert a cuvette of distilled water into the dark box and click to complete the Baseline

Scan (I0).

3. Insert the highest concentration sample of the chemical and click Auto-Find to autonomously detect the optimal LED wavelength (λmax).

4. Sequentially input known concentration values, insert the corresponding samples, and

click Measure & Plot to build the calibration curve.

3 Option B: Measure Unknown Concentration

Use this mode to calculate the concentration of an unknown sample.

1. Select the target chemical from the Load Saved Chemical Database dropdown

menu.

2. Insert distilled water and scan to establish the baseline.

3. Insert the unknown sample and click Measure Unknown Concentration.

4. The system will display the calculated concentration utilizing the accurate algorithm

(Linear Interpolation).

4 Option C: Identify Unknown Chemical

Use this mode to identify an unknown dye of a known concentration.

1. Click Start Identification Mode.

2. Insert distilled water and complete the baseline scan.

3. Enter the known concentration value (in mM) of the unidentified sample into the text

field.

4. Insert the sample and click Insert Sample & Find Solution. The system will

instantly display the identified chemical name based on database cross-referencing.

5.3YouTube Link

Here is the full step by step video of the full project demonstration.

Here is the calibration video - https://www.youtube.com/watch?v=_04mUHqsG0M

Here is the concentration measurement and dye identification video - https://www.youtube.com/watch?v=Q4ZG1lZzpH8

6 Project Management and Cost Analysis (PO(k))

6.1 Bill of Materials

S.N.	Component / Material	Quantity	Unit Price (BDT)	Total Cost (BDT)
1	Arduino UNO R3 (Microcontroller)	1	550.00	550.00
2	OPT101 Photodiode Sensor Module	1	900.00	900.00
3	16x2 LCD Display (with I2C module)	1	200.00	200.00
4	RGB LED	1	60.00	60.00
5	Custom 3D Printed Housing	1	450.00	450.00
6	Breadboard & Jumper Wires	1 Set	120.00	120.00
7	Chemical Dyes (4x 100ml Bottles)	4	162.50	650.00
8	Cuvettes & Distilled Water	1	100.00	1500.00
9	Miscellaneous (Adhesives, Screws, Tape)	1 Set	100.00	100.00
			Total Project Cost:	**4630.00**

7 Future Work (PO(l))

While the current prototype successfully demonstrates high-precision analysis, future developments will focus on commercial-level upgrades:

- **Hardware Sensitivity:** Replace the internal 10-bit ADC and OPT101 sensor with a 16-bit external ADC (e.g., ADS1115) and a high-dynamic-range digital sensor (e.g., TSL2591) to accurately measure at extreme high and low concentrations.
- **Optical Precision:** Upgrade the broad-spectrum LEDs to narrow-band optical filters or a servo-driven diffraction grating to allow continuous, highly accurate 1nm spectrum sweeps.
- **Machine Learning Integration:** Implement predictive Machine Learning models (such as lightweight Neural Networks) to replace basic linear interpolation, autonomously compensating for complex non-linear light scattering.
- **Standalone Portability:** Decouple the system from a PC by integrating a Raspberry Pi with a touchscreen interface, or by using an ESP32 Bluetooth module to transmit live data to a custom mobile application for field testing.

8 References

1. https://microbenotes.com/colorimeter-definition-principle-parts-uses-examples/

2. https://www.hunterlab.com/blog/lab-vs-lch-coordinates/

3. https://www.scitekglobal.com/how-does-a-colorimeter-work.html